Agents of Intelligence in Practice: A Multi-Agent Model Recommendation System for the National Data Platform

Tainã Coleman*, Hena Ahmed*, Ishayu Ghosh*, Ismael Perez*,
Kai Lin*, Mai Nguyen*, Tara Javidi*, İlkay Altintaş*
**University of California San Diego*, La Jolla, California, USA
{t1coleman, h7ahmed, ighosh, i3perez, klin, mhnguyen, tjavidi, ialtintas}@ucsd.edu

***Abstract*—Domain scientists are increasingly incorporating machine learning and artificial intelligence (ML/AI) into their research workflows, but many lack the computational expertise to choose an appropriate model for their data and research tasks. This paper presents the first concrete instantiation of the *Agents of Intelligence* framework: a standalone *multi-agent model recommendation system* that identifies appropriate machine-learning models given a user-supplied dataset. The agent makes decisions by leveraging academic literature search, open-source model availability, and similarity-based caching of prior recommendations. Our agent of intelligence is delivered as an independent service exposing a web interface, a Model Context Protocol (MCP) server, and a REST/Python API, and integrates with the National Data Platform (NDP) through those public interfaces. We describe our multi-stage pipeline, a 38-dimensional dataset fingerprint that powers a retrieval-augmented cache, and an empirical evaluation across three LLMs (gpt-4o-mini, Gemini, Qwen3) with three datasets (EuroSAT, IMDB, SurGen). We refined our algorithm by introducing three novel components to it: (i) a citations-per-year recency-aware ranking score, (ii) a HuggingFace popularity-weighted aggregator, and (iii) a compound-name decomposition step that normalizes across stylistic differences in how different LLMs name extracted models. With those changes, our agentic system achieves Hit@5 of 100 % on every (dataset, LLM) cell evaluated; the same ablation moves overall Hit@5 from 80 % to 100 % and roughly doubles mean precision. Family-level recall against PapersWithCode-derived ground truth reaches 0.45 overall and 0.57 on IMDB. We also show a cross-dataset cache hit (similarity of 0.99), obviating the need to run the full pipeline for structurally similar datasets. Together, these results turn the previously conceptual "Model Agent" role into a measured, deployable artifact, and identify reusable lessons to advance the broader Agents of Intelligence vision.**

***Index Terms*—Artificial Intelligence, Scientific Workflows, Agentic Systems, Model Selection, AI for Science**

I. Introduction

Artificial Intelligence has become an everyday tool in domain science practices, but the barriers posed by this rapidly evolving technology are prevalent across fields. Questions such as *"which model should I use for my data?"* remains a recurring bottleneck. Our prior interview study with eleven postdoctoral researchers at UC San Diego [4] found that almost every participant could articulate their research questions, yet few had a systematic way to answer them with machine learning. Model selection lives in the tacit knowledge of an ML practitioner, scattered across workshop tutorials, ad-hoc Slack threads, and hard-won failures. Domain scientists, by contrast, typically arrive with a clean dataset and a deadline.

In [4] we proposed *Agents of Intelligence*: a conceptual framework of modular, human-in-the-loop AI agents that embed expert heuristics into scientific workflows implemented on the National Data Platform (NDP) [1]. That framework delineates task categories for AI agents, spanning data sourcing, ingestion, curation, access, modeling, and analysis, each of which addresses a recurring pain point identified in our interviews. Stage 5, the *Model Agent*, was scoped at the conceptual level: "suggest architectures based on data modality, propose pretrained models or fine-tuning strategies, recommend loss functions and evaluation metrics. While that paper offered an overview of the framework implementation was left as future work.

This paper presents the first concrete instantiation of an Agent of Intelligence: a working multi-agent framework for Model Recommendation that takes a dataset descriptor (a local path, a URL such as a HuggingFace dataset page, or simply a name plus a task keyword) and returns a ranked list of candidate models with reasoning, paper provenance, HuggingFace availability, and a cached similarity-based shortcut for previously-seen datasets. Our key design decision was to build the agent as *a standalone framework that integrates with NDP through public APIs and an MCP server*, rather than as an internal NDP component. We also support a plethora of OpenAI SDK-compatible LLMs and tool APIs, creating a BYOK (Bring Your Own Key) system. These choices keep the agentic system reusable across deployment environments and platforms, transparent to audit, and consistent with the "composability" design principle articulated in the original framework [4]. Note that we refer to the work in this paper as "agent" or "multi-agent system/framework" interchangeably due to its unique classification as an Agent of Intelligence composed of multiple agents.

Contributions. (1) A standalone, deployable multi-agent Model Recommendation framework that connects to NDP via MCP and APIs, and is also usable through a chat user interface (UI) or as a Python library (Sections III–IV). (2) A 38-dimensional dataset fingerprint and a retrieval-augmented

caching layer that returns recommendations without LLM calls when a structurally similar dataset has been seen before (Section V). (3) Two algorithmic improvements that we identify and ablate, a citations-per-year recency-aware citation score and a compound-name decomposition step, that together raise overall Hit@5 from 80 % to 100 % across all (dataset, LLM) cells while improving mean precision P@5 from 0.16 to 0.31 (Sections VIII–VIII-D). (4) A second concrete agent instantiation (Section VI) demonstrating that the standalone-service-plus-MCP pattern generalizes beyond model recommendation.

II. Background and Related Work

Automated machine learning. AutoML systems such as Auto-sklearn [7], AutoGluon [6], and TPOT [19] automate model selection by training and benchmarking many candidate models on a user's data. They are most effective on tabular data and require the data to be loaded and labeled, and produce a trained model rather than a recommendation. Our agent operates one step earlier: given only metadata about a dataset (and optionally a small sample), it identifies what kind of model the academic literature *recommends* for that task. The two are complementary; AutoML can be invoked with the agent's top suggestions as a starting set.

Agentic LLM systems and RAG. Frameworks such as LangChain [15] and AutoGPT [8] provide scaffolding for LLM-driven multi-tool reasoning, and domain-specific agents such as ChemCrow [2] pair an LLM with APIs used in chemistry research. Our agent follows this lineage but applies the pattern to ML model selection: the LLM plans the search query and parses paper abstracts, while deterministic external services (Semantic Scholar, OpenAlex, arXiv, HuggingFace Papers, PubMed) supply the evidence. Because these services span many fields rather than a single discipline, the framework remains domain-agnostic, applying the same pipeline to recommendations across arbitrary research areas. The dataset fingerprint cache is a form of retrieval-augmented generation [12] in which retrieval is driven by a similarity index built from previously served recommendations rather than from text. By reusing prior recommendations through this index, the cache avoids redundant LLM calls and external queries, making the framework more efficient as usage grows.

Scientific gateways and recommendation services. Science gateway frameworks such as HUBzero [16], Apache Airavata [14], and the Neuroscience Gateway [22] established a data-plus-compute foundation with community-specific access to datasets and HPC resources, and efforts such as Open Science Chain [21] and the University of California San Diego's OpenForest4D [18] that added provenance tracking and AI methods for domain-specific analysis. More recent platforms target the AI/ML lifecycle directly: AI4EOSC [10] is a federated cloud platform for scientific AI within the European Open Science Cloud, and DeepForge [3] offers a visual environment for deep learning workflows. Jupyter AI [**?**] integrates frontier-model agents into JupyterLab via the Agent Client Protocol and an embedded MCP server, and is deployable on shared research infrastructure such as the National Data Platform [1]. NDP sits at the intersection of these directions, and the Agents of Intelligence framework contributes a domain-grounded agent layer in which each deployed capability is traceable to a specific obstacle identified in our prior interview study with domain scientists.

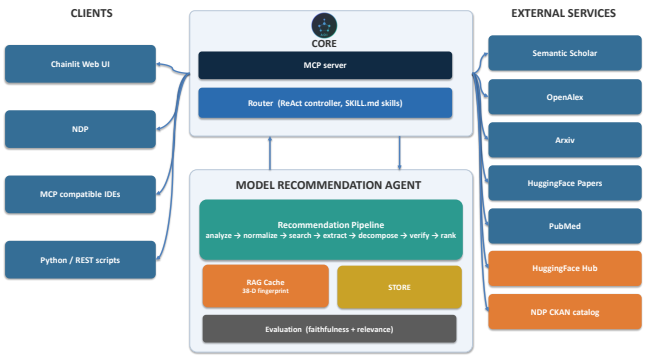


Fig. 1: Architecture of an AoI System. Illustrated here is the AoI pertinent to the Model Recommendation System, which runs as a standalone system that integrates with NDP and other clients via public interfaces (MCP, REST/HTTP) and external services (APIs). Internally, it comprises a multi-stage research pipeline with a retrieval-augmented cache.

III. System Architecture

Figure 1 shows the architecture. The multi-agent system is a standalone Python service exposing three concurrent entry points: a **Chainlit web UI** (`chainlit_app.py`) for direct end-user interaction with streaming responses and file upload; a **Model Context Protocol server** [11](`mcp_server.py`) that publishes the agent's tools to any MCP-compatible client (IDE assistants and, most relevant here, NDP itself); and a **Python/HTTP API** that exposes the same code paths for programmatic access. Internally, the system is built around a `BaseAgent`/Router pattern: a ReAct-style controller dispatches user requests to registered *skills*, each defined by a `SKILL.md` descriptor. The `NDPModelRec` skill is the focus of this paper; the router also hosts a data-harmonization skill (CKAN-format conversion), which is out of scope.

A **persistent storage layer** backs all three entry points and provides two functions: (i) durable retention of user data, uploaded files, and session state across requests (applicable to the Chainlit web UI), and (ii) a caching mechanism for model recommendations and dataset fingerprints which allows the controller to ground responses in previously ingested artifacts rather than re-invoking upstream tools on every turn. The layer is backend-agnostic: local deployments use **SQLite** for zero-configuration single-node operation, while cloud deployments use **PostgreSQL** for concurrent multi-user access.

Integration with NDP. A central design decision was that the agent must be *usable from NDP without being part of NDP*, for three reasons: *deployment portability* (a researcher can run the agent locally, on an institutional GPU, or on the federated NDP Kubernetes cluster); *auditability* (a service

whose interfaces are public is straightforward to test, version, and replace); and *composability* (an agent that communicates over standard protocols can be combined with future agents the same way) [4]. The integration works through three contracts. First, the agent registers a small set of MCP tools (`recommend_model`, `ensemble_recommend_model`, `analyze_dataset`, `save_recommendation`); MCP clients discover these at connection time, so any new agent added to the AoI framework becomes available without client-side code changes. Second, for NDP-managed dataset references, the agent calls NDP's CKAN-compatible catalog and treats it the same as the HuggingFace Hub, as an upstream metadata source. Third, the systems share no state: the agent maintains its own PostgreSQL/SQLite database of recommendation history and dataset fingerprints (Section V), which keeps either side replaceable.

IV. The Recommendation Pipeline

The full pipeline runs when a request misses the cache (Section V) and consists of seven stages, each implemented as an independent function, so the pipeline can be partially reused or replaced.

A. Dataset analysis and task normalization

The first stage produces a structured `dataset_analysis` record: detected modality (image, text, tabular, audio, video), file structure, presence of labels, an estimated sample count, and an inferred task label. For local paths, the analysis is file-system-based (counting JPEG files, inspecting CSV headers); for URLs, a two-step, deterministic fetcher pulls metadata (GitHub README via API, HuggingFace dataset card via HTML extraction), and an LLM call interprets that text into the same structured schema. The task-normalization stage then maps the user's free-form task description into a canonical taxonomy entry, such as `image_classification`, `sentiment_analysis`, `semantic_segmentation`. The implementation is keyword-first, LLM-second: a curated list of words triggers modality detection, and only then does an LLM map the description to a modality-specific task. We use word-boundary regular expressions for the keyword phase; an earlier substring-containment implementation classified every HuggingFace URL as `image_classification` because the keyword `face` (intended for face detection) matched inside "huggingface" (Section IX).

B. Multi-source academic paper search

We query up to five academic indices in parallel: Semantic Scholar, OpenAlex, arXiv, HuggingFace Papers, and PubMed. Each result includes a title, abstract, year, citation count, and URL. We deduplicate across sources by Digital Object Identifier (DOI) when available and by title similarity otherwise, then keep the top-N papers (default $N = 10$) by citation count after filtering for recency.

C. LLM-based model extraction

A second LLM call reads the retrieved paper abstracts and emits a JSON list of *(model_name, architecture, paper_index, confidence, description)* tuples. The prompt explicitly asks for both state-of-the-art and classical baselines, although Section VIII shows that the LLMs differ substantially in how they comply.

D. Compound-name decomposition

After extraction, we apply a deterministic post-processing step that we call *compound-name decomposition*. Many extracted model names combine a known base architecture with a domain prefix, suffix, or modifier (e.g., BioBERT, RoBERTa-large, Mask R-CNN). For each extracted name, we scan for occurrences of known atomic-architecture tokens (BERT, RoBERTa, LSTM, CNN, ResNet, ViT) and, for each match, emit an additional canonical mention that records only the matched token; the original mention is preserved, and both inherit the same provenance of the paper.

This step is small but consequential. Different LLMs extracted models differently: Gemini returns canonical atomic names like `BERT`; gpt-4o-mini returns descriptive paraphrases like `RoBERTa-based hybrid model`; qwen3 returns chained architectures like `BERT-BiLSTM-BiGRU-CNN`. The evaluator and the HuggingFace lookup both expect canonical names, so whether the extraction LLM returns BERT, BioBERT, or BERT-base-uncased for the same underlying architecture determines whether the lookup succeeds. With decomposition, all three LLMs produce comparable atomic candidate sets that aggregation merges with any standalone mentions from other papers. Section VIII-D quantifies the impact.

E. Aggregation, baseline injection, HuggingFace verification

The decomposed-and-aggregated mention list is enriched with two further candidate sources. First, baseline models appropriate for the dataset's *complexity tier* (computed from sample count and modality) are injected: Random Forest / XGBoost for tabular tasks, ResNet-18 / MobileNetV3 for small image datasets, TF-IDF or DistilBERT for text, and so on. Second, the top-10 candidates by preliminary citation rank are verified against the HuggingFace Hub. Verification attaches the canonical `model_id`, download count, and like count when a match is found, and flags the model as unavailable when not.

F. Ranking

The final ranking combines three normalized signals: citation score, HuggingFace popularity and extraction confidence. The citation score is calculated by

$$\text{cite_score} = \log\left(1 + \frac{c}{\max(1,\, y_{\text{now}} - y)}\right), \qquad (1)$$

where c is the cited paper's citation count and y its publication year. This is the recency-aware variant introduced in Section VIII-D: a five-year-old paper with 100 citations and a one-

year-old paper with 100 citations are no longer treated equivalently. The HuggingFace (HF) popularity score is a log-scaled combination of downloads and likes, capturing community adoption as a proxy for deployment readiness. The extraction-confidence score maps the LLM's `high`/`medium`/`low` self-rating to $\{1.0, 0.6, 0.3\}$, down-weighting models that the LLM is hedging on. After min-max normalization, the three signals are combined linearly with weights 0.4, 0.5, 0.1 on citations, HF popularity, and confidence, respectively. The HF weight is the largest because HuggingFace adoption is the most direct signal of real-world usability; citations contribute roughly as much in aggregate, but with more noise (older foundational papers can dominate newer Transformer-era work, even with the recency correction, and the LLM's self-reported confidence is the least reliable signal of the three and acts only as a tiebreaker. The weights were chosen by inspection on a held-out set; Section VIII-D ablates the structural choices (the recency correction and the HF/citation balance) but does not sweep the weights themselves.

G. Reasoning generation and evaluation

For each ranked model, we generate a short human-readable rationale that combines dataset-context ("your dataset is small ($\sim$1 000 samples), simpler models often outperform deep learning at this scale"), computational cost, paper support, and HuggingFace availability. We also compute two intrinsic quality scores: *faithfulness*, a deterministic heuristic based on how many of the retrieved papers actually mention the recommended model weighted by the LLM's confidence; and *relevance*, an LLM-judged score evaluating whether each recommendation is appropriate for the declared task.

V. Similarity-Based RAG Cache

The full pipeline costs three to four LLM calls and several HTTPS calls round-trips per request, taking anywhere from twenty seconds (gpt-4o-mini) to fifteen minutes (qwen3 hosted on a research cluster). Many requests in practice arrive for datasets that the system has effectively seen before under a different name, so we precede the pipeline with a similarity-based cache.

A. Dataset fingerprinting

Each request produces a 38-dimensional fingerprint: modality one-hot encoding (5 dimensions: image, text, tabular, audio, video), task-type one-hot encoding (28 dimensions across our task taxonomy), and five scalar features—`has_labels` (binary), complexity tier (ordinal $\in [0, 1]$), classical-viability (binary), log-scaled sample count, log-scaled feature count. The vector is computed deterministically from the dataset analysis and the normalized task label, so identical metadata yields identical fingerprints regardless of the requesting LLM.

B. Similarity scoring and retrieval

Given a query fingerprint q and a stored fingerprint s, let m_x denote modality and $\tau(x)$ denote task. The similarity is

$$\begin{aligned}\mathrm{sim}(q,s) = 0.30 \cdot \cos(q,s) + 0.35 \cdot \mathbf{1}[m_q = m_s] + \\ 0.35 \cdot \tau(q,s),\end{aligned}$$

$$\tau(q,s) = \begin{cases} 1.0 & \text{if an exact task match} \\ 0.6 & \text{if the same task family} \\ 0.0 & \text{otherwise} \end{cases} \tag{2}$$

For the task family, we refer to the following: image classification, object classification, text classification, NLP, or tabular. When $m_q \neq m_s$ we set $\mathrm{sim}(q,s) = 0$ regardless of other terms. A request whose top-1 retrieved entry exceeds a threshold (default 0.80) is served directly from the cache.

C. Cross-dataset cache hit: a worked example

After populating the database with 30 fresh runs over EuroSAT and IMDB (Section VIII), we issued a recommendation request for the SurGen digital-pathology dataset [17]. SurGen had no exact entry in the database, but its fingerprint matched an existing `cows` entry [] (an image-classification dataset of $\sim$100 cow and buffalo images) at a similarity 0.99, and the top-five EuroSAT entries at 0.96. The cache served the cached recommendations in well under one second across all three LLM clients, returning a top-five that all three LLMs would have otherwise spent collectively over thirty minutes producing. This is exactly the cross-domain transfer behavior we wanted: the fingerprint encodes "image classification, moderate scale, labeled, deep-learning viable," and that signal is genuinely shared across satellite imagery, pathology slides, and farm-animal photographs.

VI. ARGUS: A Deep Agent

This Model Recommendation System is structured as a multi-agent pipeline: the data-analysis, normalization, and model-extraction stages themselves are executed by LLM-driven agents coordinated by the router described in Section III. To deliver a complete tool to the end user, we attach a downstream *deep agent* that trains and evaluates the ranked models on the user's data using natural-language prompts alone.

To address the full scope of the obstacle identified in our prior interview study, domain scientists who can articulate a modeling task but cannot implement the resulting workflow in code, we attach a downstream execution agent, **ARGUS** (Agentic Research and Generative Understanding for Science), to the Model Recommendation system. ARGUS consumes the recommender's ranked output and autonomously executes the remaining machine-learning pipeline end-to-end, including data acquisition, fine-tuning, evaluation, and comparative analysis, through natural-language prompts issued within a single Jupyter notebook. Together, the deep agent and ARGUS form an integrated pipeline that spans from dataset description to trained, evaluated model without requiring the user to write code or select hyperparameters.

ARGUS operates as an in-kernel notebook agent whose cell magic `%%ask` dispatches each natural-language prompt to an LLM-driven controller. The agent plans a sequence of actions, generates the corresponding Python code, and executes it inside the user's kernel, drawing on a library of pre-installed skills that encode available APIs and domain conventions. For instance, the Model Recommendation Agent is exposed to ARGUS as one such skill. Because the agent runs in the user's kernel while the dataset resides on the user's local filesystem, an out-of-process MCP server would lack direct read access; accordingly, the integration binds to the recommender's Python API, one of the three public surfaces described in Section IV. The skill template invokes `recommend_models(...)` with the dataset path and task description, and the returned ranked list is treated as authoritative input to subsequent cells. Crucially, no modifications were made to the recommender to support this integration; its existing Python API was sufficient. This outcome constitutes the first empirical confirmation that the three-surface, no-shared-state architecture of Sections IV–V generalizes beyond its original use case: a user with fundamentally different runtime constraints and access to a local filesystem with in-kernel execution can interact with the recommender through a single public interface, yielding an end-to-end capability from two independently deployable services.

We validated the integrated pipeline on EuroSAT using a five-cell natural-language session: (i) retrieve the EuroSAT RGB archive and characterize class distribution and representative samples; (ii) query the Model Recommendation Agent for the two highest-ranked models; (iii) fine-tune and evaluate the top recommendation on the full 27,000-image dataset; (iv) repeat the previous step for the second recommendation; (v) generate a structured comparison report. The recommender's top two candidates, ViT-Base (rank 1, score 0.60) and ResNet-50 (rank 2, score 0.58), are consistent with the cross-LLM consensus reported in Section VIII. Fine-tuning both models on an identical 80/10/10 partition and evaluating on the held-out 2,700-image test set, the comparison report ranked ViT-Base substantially above ResNet-50, corroborating the recommender's ordering on empirical evidence. All intermediate artifacts, training scripts, model checkpoints, evaluation metrics, and diagnostic plots were persisted to a per-notebook output directory. The objective of this experiment is to demonstrate that the recommender's output, when consumed by an autonomous notebook agent, is sufficient to drive a complete and reproducible ML workflow for a user without implementation expertise.

VII. Evaluation

We evaluate the agent on two datasets across two modalities, using three LLM backends, with five repeated runs per (dataset, LLM) cell. Table I summarizes the experimental setup; A third dataset, SurGen, is used as a cache-evaluation case (Section V).

Ground truth. For each dataset, we constructed a ground-truth set of expected top models from the corresponding PapersWithCode [20] leaderboard and supplementary domain-specific benchmarks (IMDB's is dominated by BERT-family models and TF-IDF baselines; EuroSAT's by ResNet, ViT, and Swin variants derived from the published paper and accompanying dataset. It is intentionally over-inclusive (including name-level and architectural-family entries) to support both exact and family-level matching.

TABLE I: Datasets and LLM backends used in the evaluation. NRP denotes the National Research Platform's Nautilus inference service.

Datasets		
Name	**Modality**	**Task**
EuroSAT [9]	Image	10-class satellite land-cover
IMDB (Stanford NLP) [13]	Text	Binary sentiment
SurGen [17]	Image	Digital-pathology classification
LLM backends		
Name	**Provider**	**Host**
gpt-4o-mini	OpenAI	OpenAI API
gemma	Google (Gemini)	NRP Nautilus
qwen3	Alibaba Cloud	NRP Nautilus

Metrics. Each per-run output is scored on the following metrics: *Precision@K* (how much of it was returned is actually correct) and *Recall@K* (how much of what it should have found, was actually found) for $K \in \{1, 5\}$ against the ground truth at both exact-name and architectural-family levels; *Hit Rate@K* (whether at least one ground-truth model is in the top-K); *Faithfulness* (paper-mention coverage weighted by extraction confidence); *Relevance* (LLM-judge score); *Cross-LLM consistency* (pairwise Jaccard, or overlap, of all top-5 rankings across LLMs, plus per-LLM *stability* across repeats); and *Cost* (tokens, LLM latency, wall-clock).

VIII. Experimental Results

We executed 30 pipeline runs (2 datasets $\times$ 3 LLMs $\times$ 5 repeats) plus 3 cache-hit runs for SurGen, all on the same code revision and ground truth. Cache lookup was disabled for the 30 main runs so each one exercised the full pipeline.

A. Headline result: Hit@5 = 100 % across every cell

The most user-facing metric is Hit@5, the probability that a request returns at least one expert-validated model in its top-5. After the algorithmic refinements ablated in Section VIII-D, this metric is **100 % in every (dataset, LLM, run)** combination tested: 30/30 runs, 0/30 misses. Table II reports the full breakdown at both the exact-name and architectural-family level. We report both because the curated PapersWithCode ground-truth lists are intentionally narrow (3 entries for EuroSAT, 14 for IMDB); a strict exact-name match is unkind to a recommender that legitimately surfaces equivalent architectures (DistilBERT when the GT lists BERT). Family-level matching collapses each model to its architectural family and is the more faithful measure of whether the agent is recommending the *right kind* of model.

TABLE II: Per-cell Hit@5, P@5, and R@5 (exact / family) for the final agent configuration. Each cell averages five repeated runs. **Hit@5 is 100 % across the board**; family-level recall on IMDB reaches 0.571.

LLM	Dataset	Hit@5	P@5 (exact/fam)	R@5 (exact/fam)
gpt	EuroSAT	5/5	0.400 / 0.400	0.333 / 0.333
gpt	IMDB	5/5	0.240 / 0.240	0.243 / 0.571
gemma	EuroSAT	5/5	0.400 / 0.400	0.333 / 0.333
gemma	IMDB	5/5	0.200 / 0.200	0.143 / 0.571
qwen3	EuroSAT	5/5	0.400 / 0.400	0.333 / 0.333
qwen3	IMDB	5/5	0.200 / 0.200	0.143 / 0.571
Overall (n=30)		**30/30**	0.307 / 0.307	0.255 / **0.452**

B. Interpreting precision and recall

P@5 of 0.307 should not be read as "70 % of recommendations are wrong." R@5 is *mechanically capped* by the size of the ground-truth list $|G|$: for IMDB with $|G| = 14$, the achievable R@5 ceiling is $5/14 = 0.357$; we reach 0.243 (exact) and 0.571 (family). The family-level number exceeds the exact ceiling because multiple family-equivalent entries (BERT, BERT-base, DistilBERT) collapse to a single "bert" family bucket that a single recommended BERT satisfies. The agent's other top-5 picks are not random either: they include sensible non-leaderboard models (1-D CNN and BiLSTM for IMDB) that domain practitioners commonly try but that PapersWithCode does not enumerate. Hit@5 is the deployment-relevant metric; family-level recall of 0.452 overall (0.571 on IMDB) is the more honest summary of how often the agent is in the right architectural neighborhood.

C. Cross-LLM consistency, stability, and cost

Table III reports per-LLM stability (Jaccard between repeated runs) and cross-LLM consensus (Jaccard between LLM pairs after decomposition). gemma is highly deterministic; gpt-4o-mini is also stable; qwen3 shows the most variability. Cross-LLM consensus is highest on EuroSAT (ResNet-50 and ViT in all three LLMs' top-5); on IMDB the consensus is BERT and CNN. Table IV reports cost: there is a two-order-of-magnitude gap between gpt-4o-mini and qwen3 in both latency and tokens, with gemma in the middle. Faithfulness and relevance are not strongly correlated with cost, suggesting the gating bottleneck for quality is extraction style and paper-search variability, not raw model capability (Section IX).

TABLE III: Per-LLM stability across 5 repeats and pairwise cross-LLM consensus (Jaccard of top-5).

	EuroSAT	IMDB
Per-LLM stability (5 repeats)		
gemma	1.00	1.00
gpt-4o-mini	0.69	0.87
qwen3	0.56	0.60
Avg pairwise cross-LLM Jaccard (top-5)		
all pairs	0.35	0.37

The 3 SurGen runs all hit the cache with similarity 0.99, returning in sub-second wall-clock time. For a deployed system serving a research community with overlapping interests, the cache hit rate rises sharply after the first few hundred unique structural fingerprints are populated, since most ML tasks fall into a small number of (modality, task, scale) buckets.

TABLE IV: Mean cost per run by LLM (EuroSAT and IMDB pooled, n=10 each).

LLM	Tokens	LLM latency (s)	Wall (s)
gpt-4o-mini	2,157	10.1	23.2
gemma	2,235	29.6	45.0
qwen3	17,139	788.9	797.2

D. Ablation: Ranker Design

The headline numbers were not the agent's out-of-the-box behavior: an earlier ranker produced substantially worse results on IMDB. We ablate three configurations:

- **C0 (citation-heavy)** is the original ranker, with $\text{cite_score} = \log(1 + c) \cdot (1 + 1/\text{age})$ and weights $(0.4, 0.3, 0.1)$ on citations, HF popularity, and fit.
- **C1 (recency + HF)** replaces the citation score with $\log(1 + c/\text{age})$ and raises the HF weight from 0.3 to 0.5; citations-per-year prevents older foundational papers from automatically dominating newer Transformer-era work, and the larger HF weight lets a well-known architecture (BERT, 59M downloads) outrank a niche match (a CNN paper with 129 citations).
- **C2 (recency + HF + decomposition)** adds the compound-name decomposition of Section IV-D.

Table V reports the headline metrics for all three configurations. The first two columns (C0, C1) reflect changes to how candidates are *ranked*; the third (C2) changes *what candidates exist* to rank at all, by emitting canonical atomic names from compound or paraphrased extractions (Section IV-D). This distinction explains the result: C1's recency-aware citation score and increased HF weight rescue one previously-failing query (Hit@5 $24/30 \rightarrow 25/30$), but cannot help in cells where the right architecture is absent from the candidate pool. C2 introduces those missing canonical candidates and closes the remaining gap entirely ($25/30 \rightarrow 30/30$).

The "worst cell" row tracks the single (LLM, dataset) combination that motivated this analysis: gpt-4o-mini on IMDB. Under C0 it scores $0/5$ because gpt-4o-mini consistently returns names like `RoBERTa-based hybrid model` rather than `RoBERTa`; C1's better ranking rescues exactly one query out of five; C2's canonical decomposition rescues all five. Reporting only an averaged Hit@5 would have hidden this systematic failure inside a single cell and made C2 look like a marginal improvement instead of a structural one.

The family-level rows rule out the alternative explanation that C2's gains are a relabeling effect (different surface names collapsing to the same architectural family at evaluation time). If that were the case, exact-name metrics would improve but family-level metrics would not. Instead, family R@5 climbs at every step ($0.338 \rightarrow 0.357 \rightarrow 0.452$), meaning each ablation brings genuinely new architectural families into the top-5.

TABLE V: Ablation over the same 30-run dataset, at exact-name and architectural-family granularity. C2's compound-name decomposition is the largest single contributor: it rescues the worst cell (gpt-4o-mini on IMDB) from $0/5$ to $5/5$, lifts mean exact P@5 from 0.173 to 0.307, and lifts family-level R@5 from 0.357 to 0.452.

Metric	C0 (citation-heavy)	C1 (+ recency + HF)	C2 (+ decomposition)
Hit@5	24/30 (80 %)	25/30 (83 %)	**30/30 (100 %)**
P@5 (exact)	0.160	0.173	**0.307**
P@5 (family)	0.160	0.173	**0.307**
R@5 (exact)	0.210	0.221	0.255
R@5 (family)	0.338	0.357	**0.452**
Worst cell (exact)	0/5	1/5	**5/5**

Exact and family P@5 are identical in this experiment because each ground-truth family has at most one canonical exact-name entry that the agent surfaces; the divergence between exact and family appears on the recall side, where families with multiple exact-name members (e.g., the BERT family with BERT, BERT-base, DistilBERT) account for the extra credit.

Why decomposition matters. The 0 % cell in C0 and C1 is informative. On IMDB, gpt-4o-mini's extraction typically returns names like `RoBERTa-based hybrid model` rather than `RoBERTa`. The downstream HuggingFace lookup and ground-truth matcher both expect canonical names; "RoBERTa-based hybrid model" has zero HF downloads (it is not a real model on the Hub) and is at best a partial substring match against the ground-truth BERT-family entries. By emitting an additional canonical `RoBERTa` mention from the same paper provenance, aggregation merges across paper sources and the ranking sees a candidate that the paper genuinely supports *and* has 19 M HF downloads. The same effect rescues qwen3's chained `BERT-BiLSTM-BiGRU-CNN` extractions.

IX. Discussion

LLMs differ in extraction *style*, not capability. Before decomposition, gemma is at 100 % Hit@5 on IMDB, qwen3 at 80 %, and gpt-4o-mini at 0 %, despite all three being entirely capable of identifying BERT/RoBERTa as relevant. The difference is in the naming convention: gemma emits atomic taxonomic names, gpt-4o-mini emits descriptive paraphrases, qwen3 emits ensembles. Any LLM-driven research agent that consumes structured names downstream needs to either adopt a canonization step or aggressively constrain the LLM's output schema.

Failure modes are silent. The keyword-substring bug in Section IV is a classic example of a silent LLM-system failure: every IMDB request happily produced a fully-formed top-5 of image-classification models, with no error and no warning. Traditional unit testing barely addresses this class of bug; we now treat per-(dataset, LLM) Hit@5 as a regression metric and flag any cell that drops below an empirical threshold.

Caching gives the framework headroom for slower, better LLMs. The ten-second vs. thirteen-minute gap between gpt-4o-mini and qwen3 is the operational reason the cache is essential, not optional. Once a fingerprint is in the cache, the slower LLM's quality contribution becomes a *one-time* cost amortized across all future requests for similar datasets. It is more affordable to have a higher-quality but slower LLM if it pays that cost once per fingerprint rather than once per user.

Implications for the broader framework. Three findings can be generalized from our study. (i) *Structured external services first; LLMs as glue.* The pipeline succeeds because deterministic services (paper APIs, HF Hub) provide most of the evidence; the LLM translates user intent into queries and extracts from free text, roles where its strengths offset its hallucination risk. (ii) *Canonization is universal.* Every Agent of Intelligence that produces structured outputs from LLM text will need an analog of compound-name decomposition for its domain (datasets, instruments, methods, units of measurement). (iii) *Fingerprint-based caching scales the framework.* Each new agent can reuse the same cache pattern over its own fingerprint schema, yielding compound benefits as agents are composed.

Why not just ask an LLM directly? The simplest baseline for this task is to skip the pipeline and ask an LLM "recommend the top models for sentiment classification on IMDB." This baseline fails along four axes around which the pipeline is built. First, *evidence*: a direct LLM answer is a single opaque generation with no provenance. The pipeline produces every recommendation alongside the academic papers it was extracted from, an HF Hub `model_id` with download and like counts, a faithfulness score measuring how many retrieved papers actually mention the model, and an LLM-judge relevance score. The user can audit each link in the chain; a bare LLM response offers no such handles. Second, *recency and availability*: LLM weights are frozen at training time, so a direct query returns whatever was canonical at the cutoff, undifferentiated from models that have since been deprecated or surpassed. The recency-aware citation score and HF verification step explicitly down-weight stale recommendations and flag models that are no longer available on the Hub, signaling that the LLM cannot produce them on its own. Third, and most importantly for the framework's intended users, *coverage of unfamiliar datasets*: a direct LLM query only works when the dataset has been memorized during training. The pipeline does not depend on dataset identity at all. It characterizes each dataset by a fingerprint (modality, sample count, label structure, inferred task, complexity tier) derived from metadata, so the data format and dataset fame are irrelevant. The SurGen example in Section V makes this concrete: a 2025 colorectal-cancer whole-slide-image dataset with essentially no presence in any LLM's training data was served sub-second from cache entries originally produced for satellite imagery and farm-animal photographs, because the fingerprint correctly identified all three as the same recommendation problem (labeled, moderately-sized, deep-learning-viable image classification). A bare LLM cannot do this. It can only retrieve what it has seen by name; it has no mechanism to recognize that an unfamiliar dataset is operationally equivalent

to a familiar one. The pipeline is, in effect, an LLM with each of its known weaknesses (stale knowledge, unverifiable outputs, inconsistent naming, and zero recall on unfamiliar inputs) delegated to a deterministic component better suited to the job.

X. Limitations and Future Work

The main limitations are: (1) we evaluate on three datasets across two modalities, all in English; multilingual or graph-structured data is untested. (2) The ground truth is PapersWithCode-biased; emerging or niche methods may be missed even when valid. (3) The fingerprint encodes *structural* similarity (modality, task, scale, complexity) but not *semantic* similarity (satellite vs. medical imagery); a domain-aware embedding could be added without changing the rest of the system. (4) We do not yet evaluate downstream model performance: a recommendation that lands BERT in the top-5 is scored equally to one that lands BERT at rank 1.

Three follow-ups are immediate. First, we will extend the agent to consume *small dataset samples* where available, allowing AutoML-style empirical evaluation of the recommended top-K on the user's actual data and feeding the resulting performance numbers back into the ranker as a fourth signal. Second, we will instantiate additional categories of agents (Data Agent or Infrastructure Agent from [4]) using the same architectural pattern to validate that the reusable parts truly generalize. Third, we plan a long-term study with the NDP user community: deployment, implicit-feedback collection (accept/ignore signals on the top-K), and adaptation of the ranker weights per discipline.

XI. Conclusion

We present the first concrete realization of an Agent of Intelligence: a standalone multi-agent Model Recommendation System that integrates with the National Data Platform via public APIs and an MCP server while remaining independently deployable. We described its architecture—a multi-stage, research-driven pipeline with a similarity-based cache that enables cross-dataset reuse—and evaluated the agent across three LLMs and three datasets. Two algorithmic improvements—citations-per-year ranking and compound-name decomposition—together raise Hit@5 from 80 % to 100 % across all (dataset, LLM) cells. A second agent (Section VI) demonstrates that the standalone-service-plus-MCP pattern generalizes. Beyond the performance metrics, our work surfaces three broadly applicable lessons: LLMs differ in extraction style and need explicit canonization, silent failures must be caught with regression metrics, and fingerprinted caches let research communities amortize the cost of high-quality but slow LLMs. The "Model Agent" role transitions from a category in a framework diagram to a measured, deployable artifact and a template that the rest of the framework can follow. With this agent, we address and provide a concrete solution to a major pain point for scientific research communities, as identified from the interviews in [5], which is the main motivation of this work.

XII. Acknowledgment

This work was supported by NSF award 2134904 for BurnPro3D, NSF award 2333609 for the National Data Platform and Schmidt Sciences. The Nautilus Kubernetes Cluster of the National Research Platform, funded by NSF 2112167 and 2120019, was also used for this research. The authors thank their team at the San Diego Supercomputer Center for their collaboration.

References

[1] National Data Platform (2024), https://nationaldataplatform.org/
[2] Bran, A.M., Cox, S., Schilter, O., Baldassari, C., White, A.D., Schwaller, P.: Chemcrow: Augmenting large-language models with chemistry tools. arXiv preprint arXiv:2304.05376 (2023)
[3] Broll, B., Whitaker, J.: Deepforge: an open source, collaborative environment for reproducible deep learning (2017)
[4] Coleman, T., Altintaş, : Toward agents of intelligence: Bridging the ai expertise gap in domain sciences. In: 2025 IEEE International Conference on eScience (eScience). pp. 460–466 (2025). https://doi.org/10.1109/eScience65000.2025.00092
[5] Coleman, T., Altintaş, : Toward agents of intelligence: Bridging the ai expertise gap in domain sciences. In: 2025 IEEE International Conference on eScience (eScience). pp. 460–466 (2025). https://doi.org/10.1109/eScience65000.2025.00092
[6] Erickson, N., Mueller, J., Shirkov, A., Zhang, H., Larroy, P., Li, M., Smola, A.: Autogluon-tabular: Robust and accurate automl for structured data. arXiv preprint arXiv:2003.06505 (2020)
[7] Feurer, M., Klein, A., Eggensperger, K., Springenberg, J., Blum, M., Hutter, F.: Efficient and robust automated machine learning. Advances in neural information processing systems **28** (2015)
[8] Fezari, M., Ali-Al-Dahoud, A.A.D.: From gpt to autogpt: a brief attention in nlp processing using dl. Preprint (2023)
[9] Helber, P., Bischke, B., Dengel, A., Borth, D.: Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification. IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing **12**(7), 2217–2226 (2019). https://doi.org/10.1109/JSTARS.2019.2918242
[10] Heredia, I., García, Á.L., Gómez, F.A., Aguirre, D., Marín, C.A., Alibabaei, K., Berberi, L., Caballer, M., Calatrava, A., Castro, P., et al.: Ai4eosc: a federated cloud platform for artificial intelligence in scientific research. arXiv preprint arXiv:2512.16455 (2025)
[11] Hou, X., Zhao, Y., Wang, S., Wang, H.: Model context protocol (mcp): Landscape, security threats, and future research directions. ACM Transactions on Software Engineering and Methodology (2025)
[12] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., et al.: Retrieval-augmented generation for knowledge-intensive nlp tasks. Advances in neural information processing systems **33**, 9459–9474 (2020)
[13] Maas, A.L., Daly, R.E., Pham, P.T., Huang, D., Ng, A.Y., Potts, C.: Learning word vectors for sentiment analysis. In: Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies. pp. 142–150. Association for Computational Linguistics, Portland, Oregon, USA (Jun 2011), https://aclanthology.org/P11-1015/
[14] Marru, S., Gunathilake, L., Herath, C., Tangchaisin, P., Pierce, M., Mattmann, C., Singh, R., Gunarathne, T., Chinthaka, E., Gardler, R., et al.: Apache airavata: a framework for distributed applications and computational workflows. In: Proceedings of the 2011 ACM workshop on Gateway computing environments. pp. 21–28 (2011)
[15] Mavroudis, V.: Langchain (2024)
[16] McLennan, M., Kennell, R.: Hubzero: a platform for dissemination and collaboration in computational science and engineering. Computing in Science & Engineering **12**(2), 48–53 (2010)
[17] Myles, C., Um, I.H., Marshall, C., Harris-Birtill, D., Harrison, D.J.: Surgen: 1020 h&e-stained whole-slide images with survival and genetic markers. GigaScience **14**, giaf086 (2025)
[18] Nandigam, V.: Collaborative research: Frameworks: Openforest4d-a web-based cyberinfrastructure platform for next generation 4d forest mapping and monitoring. NSF Award Number 2409885. Directorate for Computer and Information Science and Engineering **24**(2409885), 9885 (2024)

[19] Olson, R.S., Moore, J.H.: Tpot: A tree-based pipeline optimization tool for automating machine learning. In: Workshop on automatic machine learning. pp. 66–74. PMLR (2016)
[20] Papers with Code: Papers with code: A free resource of machine learning papers, code, datasets, evaluation tables and methods. https://paperswithcode.com/ (2024), accessed: 2026-05-14
[21] Sivagnanam, S., Nandigam, V., Lin, K.: Introducing the open science chain: Protecting integrity and provenance of research data. In: Practice and Experience in Advanced Research Computing 2019: Rise of the Machines (Learning). PEARC '19, Association for Computing Machinery, New York, NY, USA (2019). https://doi.org/10.1145/3332186.3332203, https://doi.org/10.1145/3332186.3332203
[22] Sivagnanam, S., Yoshimoto, K., Carnevale, N.T., Majumdar, A.: The neuroscience gateway: enabling large scale modeling and data processing in neuroscience. In: Proceedings of the Practice and Experience on Advanced Research Computing: Seamless Creativity, pp. 1–7 (2018)